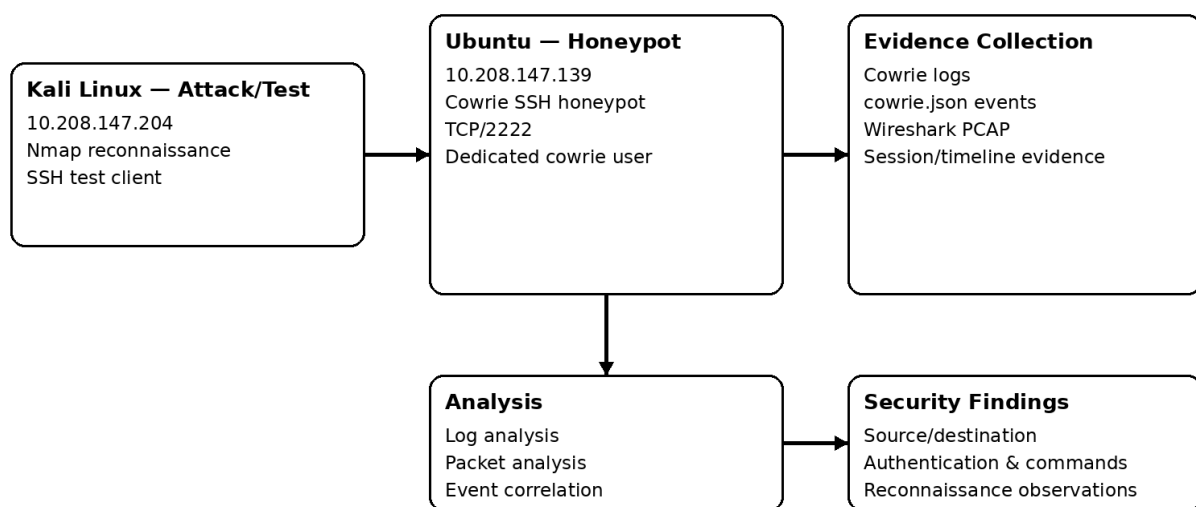


FINAL PROJECT REPORT

NETWORK HONEYPOT FOR ATTACK DETECTION AND NETWORK TRAFFIC ANALYSIS

Final Project Report — Practical Experiment

Network Honeypot — Experimental Architecture



Submitted By

[Student Name]

[Register / Roll Number]

[Department / Programme]

[College / University]

Academic Year: 2026

CERTIFICATE

This is to certify that the project entitled "Network Honeypot for Attack Detection and Network Traffic Analysis" is a record of the practical work carried out by [Student Name], [Register / Roll Number], under the guidance of [Guide Name], in partial fulfilment of the academic requirements of [Programme / Department].

Guide Signature: _____ Head of Department: _____

Date: _____ Place: _____

DECLARATION

I hereby declare that this project report is my original academic work carried out in an isolated laboratory environment. The experiments were performed only against the project-owned virtual machines, and the captured credentials were laboratory test credentials.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members and department for their guidance and support. I also acknowledge the open-source security community and the developers of Cowrie, Nmap and Wireshark for the tools used in this practical experiment.

ABSTRACT

Network honeypots are defensive security mechanisms designed to attract and observe suspicious activity in a controlled environment. This project implements a practical network honeypot laboratory using Kali Linux as the attack/test machine and Ubuntu Server as the honeypot host. Cowrie is deployed as an SSH honeypot on TCP port 2222. Controlled reconnaissance is performed using Nmap, while Wireshark captures and analyzes the corresponding network traffic. Cowrie logs provide application- and session-level evidence such as source IP addresses, authentication attempts, commands and session termination. Wireshark complements this evidence with packet-level observations including the TCP three-way handshake, SSHv2 negotiation and encrypted SSH traffic.

The experiment demonstrates how multiple evidence sources can be correlated to understand an attack sequence: reconnaissance, connection establishment, SSH negotiation, authentication, interactive commands and session termination. The work is intentionally limited to an isolated academic laboratory and focuses on safe, controlled testing.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Aim and Objectives
4. Scope of the Project
5. Background and Concept
6. System Architecture
7. Tools and Technologies
8. Experimental Methodology
9. Practical Module 1 — Lab Environment & Network Setup
10. Practical Module 2 — Ubuntu Honeypot Preparation
11. Practical Module 3 — Cowrie Installation & Configuration
12. Practical Module 4 — Controlled SSH Interaction & Logging
13. Practical Module 5 — Controlled Nmap Reconnaissance
14. Practical Module 6 — Wireshark Network Traffic Analysis
15. Practical Module 7 — Log Analysis & Event Correlation
16. Practical Module 8 — Results, Attack Analysis & Findings
17. Overall Results
18. Security Analysis and Discussion
19. Limitations
20. Future Scope
21. Conclusion
22. References
Appendix A — Evidence Mapping

1. INTRODUCTION

Modern networks are continuously exposed to reconnaissance, credential attacks and unauthorized service interaction. Traditional monitoring focuses on production systems and may miss detailed attacker behavior. A honeypot provides a controlled, instrumented service for observing suspicious interaction.

This project focuses on an SSH-based network honeypot using Cowrie. Kali Linux is used for controlled reconnaissance and SSH interaction, while Ubuntu Server hosts Cowrie. Wireshark independently captures network packets.

2. PROBLEM STATEMENT

Security analysts need practical ways to observe suspicious network activity without exposing real systems or user data. This project addresses how to build an isolated honeypot environment and correlate network traffic, authentication events, commands and logs into a coherent attack narrative.

3. AIM AND OBJECTIVES

Aim: To design and experimentally evaluate a network honeypot for detecting and analyzing controlled SSH-based reconnaissance and interaction.

Objectives: create an isolated lab; configure Kali and Ubuntu; deploy Cowrie on TCP/2222; perform controlled SSH interaction; use Nmap; capture with Wireshark; analyze text and JSON logs; correlate evidence; document findings.

4. SCOPE OF THE PROJECT

The scope is restricted to an isolated VirtualBox laboratory. The implementation focuses on SSH through Cowrie, controlled reconnaissance, packet capture and log correlation. Additional OpenCanary services were not included in the final eight practical modules. No public or third-party systems are targeted.

5. BACKGROUND AND CONCEPT

A honeypot is a deliberately monitored system or service intended to attract and record suspicious interaction. Cowrie provides a medium-interaction SSH/Telnet honeypot. Nmap is used for controlled reconnaissance and Wireshark provides packet-level analysis of Ethernet, IP, TCP and SSH traffic.

6. SYSTEM ARCHITECTURE

Network Honeypot — Experimental Architecture

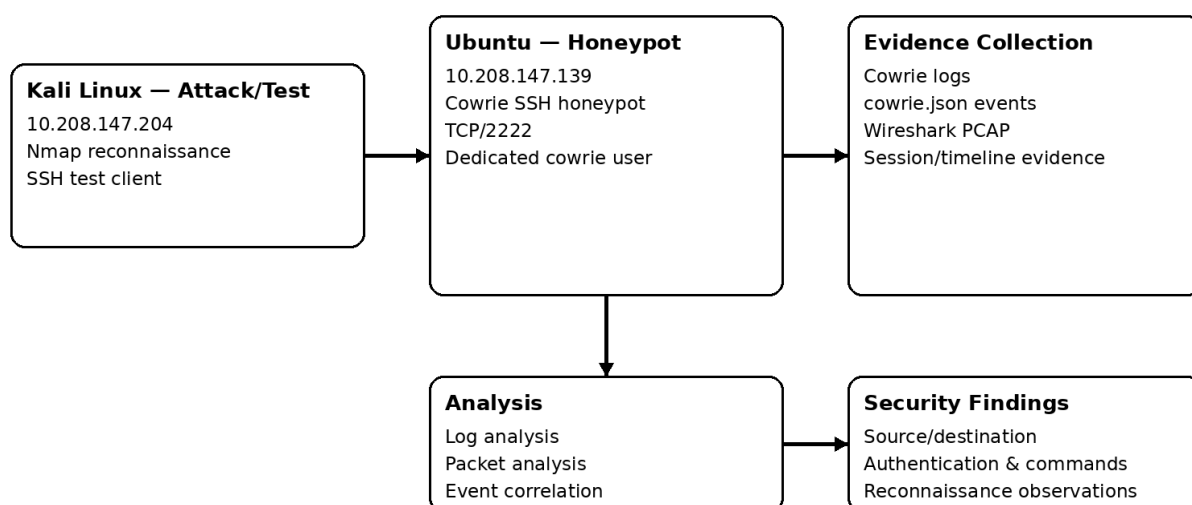


Figure 6.1 — Final experimental architecture

The architecture contains Kali Linux at 10.208.147.204 and Ubuntu at 10.208.147.139. Cowrie provides SSH on TCP/2222. Nmap generates controlled reconnaissance and SSH generates application/session activity. Cowrie logs and Wireshark captures are analyzed together.

Network addressing

Component	IP Address	Role
Kali Linux	10.208.147.204	Attack/Test Machine
Ubuntu Server	10.208.147.139	Honeypot Host
Cowrie	10.208.147.139:2222	SSH Honeypot
Network	10.208.147.0/24	Private Lab Network

7. TOOLS AND TECHNOLOGIES

Tool / Technology	Purpose
VirtualBox	Virtualization platform used to create the isolated lab VMs.
Kali Linux	Attack/test machine used for controlled reconnaissance and SSH interaction.
Ubuntu Server	Honeypot host used to deploy and operate Cowrie.
Cowrie	Medium-interaction SSH/Telnet honeypot used here for SSH activity capture.
Nmap	Network reconnaissance and port/service discovery tool.
Wireshark	Packet capture and network traffic analysis tool.
Python 3 / venv / pip	Runtime and isolated environment used to install Cowrie.
Linux shell utilities	grep, tail, ss, ip, ping, jq and related tools used for evidence collection.

8. EXPERIMENTAL METHODOLOGY

The experiment followed a staged workflow: validate the lab network, prepare Ubuntu, install Cowrie, perform controlled SSH interaction, run Nmap reconnaissance, capture traffic with Wireshark, analyze Cowrie text/JSON logs, correlate evidence and document findings.

PRACTICAL MODULE 1 — LAB ENVIRONMENT & NETWORK SETUP

Objective: Establish an isolated VirtualBox laboratory containing Kali Linux as the attack/test machine and Ubuntu Server as the honeypot machine. Verify IP addressing, routing and bidirectional connectivity.

Environment: Two VirtualBox VMs were used: Kali Linux as the attack/test machine and Ubuntu Server as the honeypot.

Addressing: Kali was verified at 10.208.147.204 and Ubuntu at 10.208.147.139 on the 10.208.147.0/24 network.

Connectivity: Bidirectional ping tests and route inspection confirmed communication between the VMs.

Evidence

A screenshot of a Kali Linux terminal window. The terminal shows the output of the 'ifconfig' command for three interfaces: 'inet', 'eth0', and 'lo'. The 'inet' interface is configured with IP 172.17.0.1 and netmask 255.255.0.0. The 'eth0' interface is configured with IP 10.208.147.204 and netmask 255.255.255.0. The 'lo' interface is configured with IP 127.0.0.1 and netmask 255.0.0.0. Below the interface information, the terminal shows the output of a ping command: 'ping -c 4 10.208.147.139'. The ping results show four successful pings with times ranging from 0.777 ms to 2.04 ms. Finally, the terminal shows the output of the 'ip route' command, which displays the default route via 10.208.147.159 dev eth0 proto dhcp src 10.208.147.204 metric 100, and two other routes for 10.208.147.0/24 and 172.17.0.0/16.

```
Session Actions Edit View Help
inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
ether de:e9:d2:73:01:c1 txqueuelen 0 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 5 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.208.147.204 netmask 255.255.255.0 broadcast 10.208.147.255
inet6 fe80::cf11:12fe:3e68:79e4 prefixlen 64 scopeid 0<link>
ether 08:00:27:5a:87:bc txqueuelen 1000 (Ethernet)
RX packets 3 bytes 776 (776.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 21 bytes 2940 (2.8 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

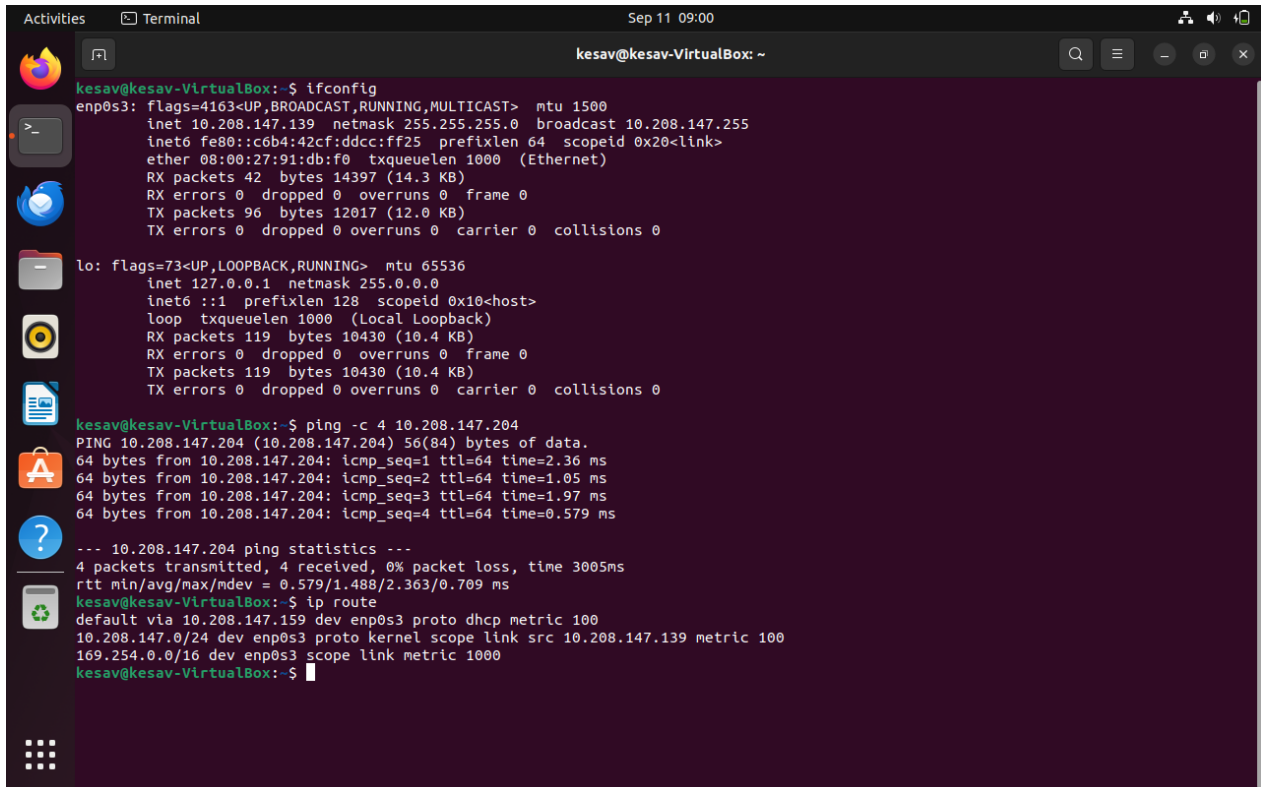
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 8 bytes 480 (480.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 8 bytes 480 (480.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@kali)-[~]
$ ping -c 4 10.208.147.139
PING 10.208.147.139 (10.208.147.139) 56(84) bytes of data.
64 bytes from 10.208.147.139: icmp_seq=1 ttl=64 time=1.71 ms
64 bytes from 10.208.147.139: icmp_seq=2 ttl=64 time=2.04 ms
64 bytes from 10.208.147.139: icmp_seq=3 ttl=64 time=1.73 ms
64 bytes from 10.208.147.139: icmp_seq=4 ttl=64 time=0.777 ms

— 10.208.147.139 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3132ms
rtt min/avg/max/mdev = 0.777/1.563/2.040/0.472 ms

(kali@kali)-[~]
$ ip route
default via 10.208.147.159 dev eth0 proto dhcp src 10.208.147.204 metric 100
10.208.147.0/24 dev eth0 proto kernel scope link src 10.208.147.204 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
```

Figure 1.1 — Kali network configuration and connectivity



```
kesav@kesav-VirtualBox: ~  
kesav@kesav-VirtualBox:~$ ifconfig  
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 10.208.147.139 netmask 255.255.255.0 broadcast 10.208.147.255  
    inet6 fe80::c6b4:42cf:ddcc:ff25 prefixlen 64 scopeid 0x20<link>  
    ether 08:00:27:91:db:f0 txqueuelen 1000 (Ethernet)  
    RX packets 42 bytes 14397 (14.3 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 96 bytes 12017 (12.0 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 119 bytes 10430 (10.4 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 119 bytes 10430 (10.4 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
kesav@kesav-VirtualBox:~$ ping -c 4 10.208.147.204  
PING 10.208.147.204 (10.208.147.204) 56(84) bytes of data:  
64 bytes from 10.208.147.204: icmp_seq=1 ttl=64 time=2.36 ms  
64 bytes from 10.208.147.204: icmp_seq=2 ttl=64 time=1.05 ms  
64 bytes from 10.208.147.204: icmp_seq=3 ttl=64 time=1.97 ms  
64 bytes from 10.208.147.204: icmp_seq=4 ttl=64 time=0.579 ms  
  
--- 10.208.147.204 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 0.579/1.488/2.363/0.709 ms  
kesav@kesav-VirtualBox:~$ ip route  
default via 10.208.147.159 dev enp0s3 proto dhcp metric 100  
10.208.147.0/24 dev enp0s3 proto kernel scope link src 10.208.147.139 metric 100  
169.254.0.0/16 dev enp0s3 scope link metric 1000  
kesav@kesav-VirtualBox:~$
```

Figure 1.2 — Ubuntu network configuration and connectivity

Module result

The isolated laboratory network was established successfully, with verified addressing, routing and bidirectional connectivity.

PRACTICAL MODULE 2 — UBUNTU HONEYPOT PREPARATION

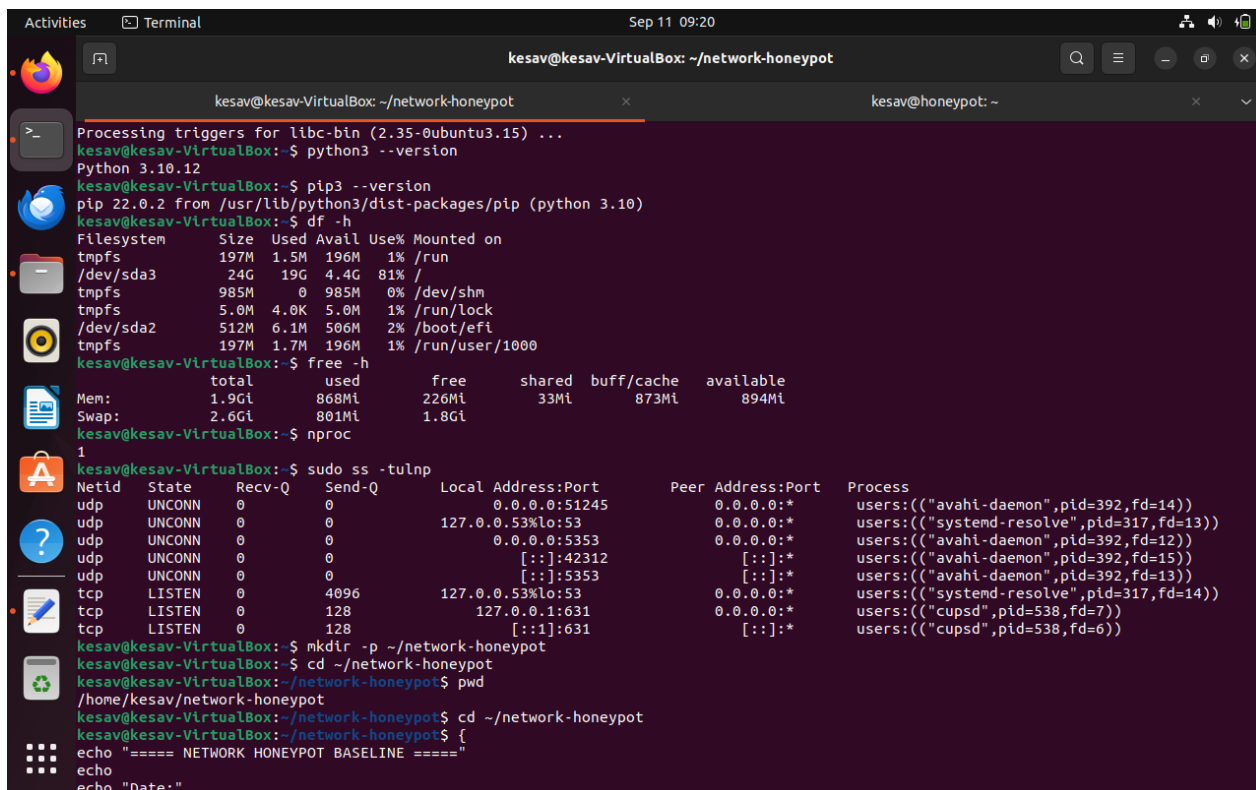
Objective: Prepare Ubuntu Server for honeypot deployment, install required utilities, create the project workspace, establish a baseline and verify the host state before deploying Cowrie.

Operating system: Ubuntu Server 22.04.5 LTS was prepared for honeypot deployment.

Host preparation: The hostname was changed to honeypot and required packages/utilities were installed.

Baseline: A project directory and baseline service-state evidence were created before honeypot deployment.

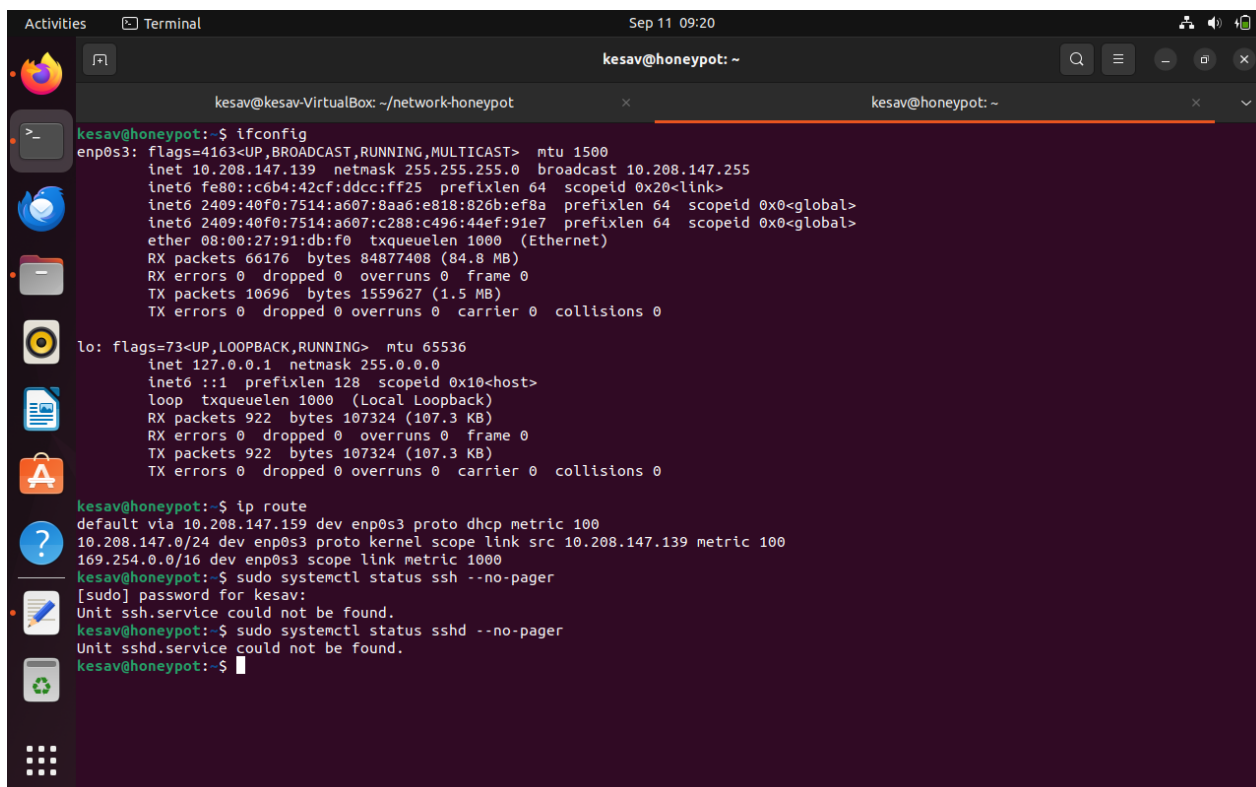
Evidence



The terminal window shows the following commands and output:

```
Processing triggers for libc-bin (2.35-0ubuntu3.15) ...
kesav@kesav-VirtualBox:~$ python3 --version
Python 3.10.12
kesav@kesav-VirtualBox:~$ pip3 --version
pip 22.0.2 from /usr/lib/python3/dist-packages/pip (python 3.10)
kesav@kesav-VirtualBox:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           197M  1.5M  196M   1% /run
/dev/sda3       24G   19G   4.4G  81% /
tmpfs           985M   0  985M   0% /dev/shm
tmpfs           5.0M  4.0K   5.0M   1% /run/lock
/dev/sda2       512M  6.1M  506M   2% /boot/efi
tmpfs           197M  1.7M  196M   1% /run/user/1000
kesav@kesav-VirtualBox:~$ free -h
             total        used        free      shared  buff/cache   available
Mem:           1.9Gi       868Mi       226Mi         33Mi         873Mi       894Mi
Swap:          2.6Gi       801Mi         1.8Gi
kesav@kesav-VirtualBox:~$ nproc
1
kesav@kesav-VirtualBox:~$ sudo ss -tulnp
Netid  State      Recv-Q    Send-Q      Local Address:Port      Peer Address:Port      Process
udp    UNCONN     0          0          0.0.0.0:51245            0.0.0.0:*               users:((("avahi-daemon",pid=392,fd=14)))
udp    UNCONN     0          0      127.0.0.0:53            0.0.0.0:*               users:((("systemd-resolve",pid=317,fd=13)))
udp    UNCONN     0          0          0.0.0.0:5353            0.0.0.0:*               users:((("avahi-daemon",pid=392,fd=12)))
udp    UNCONN     0          0          [::]:42312              [::]:*                  users:((("avahi-daemon",pid=392,fd=15)))
udp    UNCONN     0          0          [::]:5353               [::]:*                  users:((("avahi-daemon",pid=392,fd=13)))
tcp    LISTEN     0        4096      127.0.0.0:53            0.0.0.0:*               users:((("systemd-resolve",pid=317,fd=14)))
tcp    LISTEN     0        128      127.0.0.1:631           0.0.0.0:*               users:((("cupsd",pid=538,fd=7)))
tcp    LISTEN     0        128      [::]:631                [::]:*                  users:((("cupsd",pid=538,fd=6)))
kesav@kesav-VirtualBox:~$ mkdir -p ~/network-honeypot
kesav@kesav-VirtualBox:~$ cd ~/network-honeypot
kesav@kesav-VirtualBox:~/network-honeypot$ pwd
/home/kesav/network-honeypot
kesav@kesav-VirtualBox:~/network-honeypot$ cd ~/network-honeypot
kesav@kesav-VirtualBox:~/network-honeypot$ {
echo "==== NETWORK HONEYPOT BASELINE ====="
echo
echo "Date:"
```

Figure 2.1 — Ubuntu honeypot preparation and baseline checks



The terminal window shows the following commands and output:

```
kesav@honeypot:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.208.147.139 netmask 255.255.255.0 broadcast 10.208.147.255
    inet6 fe80::c6b4:42cf:ddcc:ff25 prefixlen 64 scopeid 0x20<link>
    inet6 2409:40f0:7514:a607:8aa6:e818:826b:ef8a prefixlen 64 scopeid 0x0<global>
    inet6 2409:40f0:7514:a607:c288:c496:44ef:91e7 prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:91:db:f0 txqueuelen 1000 (Ethernet)
    RX packets 66176 bytes 84877408 (84.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 10696 bytes 1559627 (1.5 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 922 bytes 107324 (107.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 922 bytes 107324 (107.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

kesav@honeypot:~$ ip route
default via 10.208.147.159 dev enp0s3 proto dhcp metric 100
10.208.147.0/24 dev enp0s3 proto kernel scope link src 10.208.147.139 metric 100
169.254.0.0/16 dev enp0s3 scope link metric 1000
kesav@honeypot:~$ sudo systemctl status ssh --no-pager
[sudo] password for kesav:
Unit ssh.service could not be found.
kesav@honeypot:~$ sudo systemctl status sshd --no-pager
Unit sshd.service could not be found.
kesav@honeypot:~$
```

Module result

Ubuntu was prepared as a clean honeypot host with the required tools and a documented baseline.

PRACTICAL MODULE 3 — COWRIE INSTALLATION & CONFIGURATION

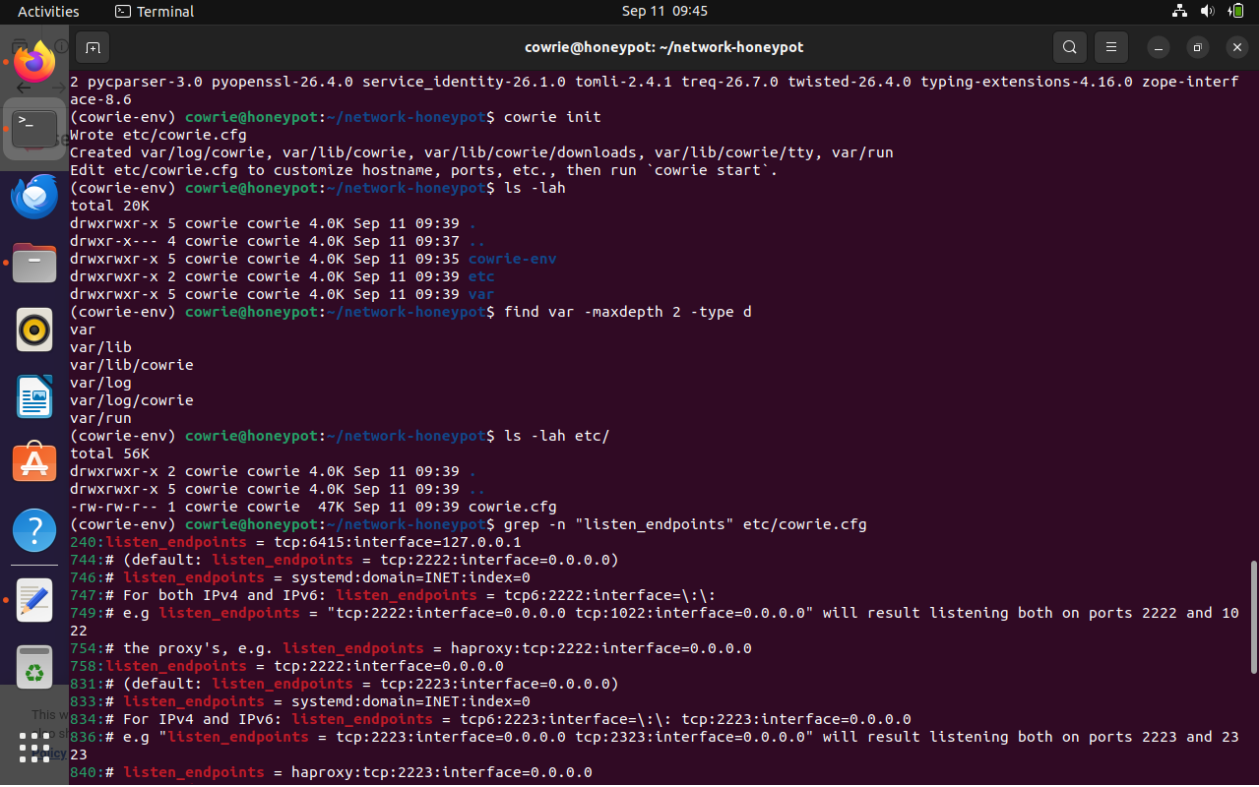
Objective: Install Cowrie in a Python virtual environment, initialize its configuration and start the SSH honeypot on TCP port 2222.

Installation: Cowrie was installed in an isolated Python virtual environment using a dedicated cowrie user.

Initialization: cowrie init created the configuration, log and runtime directories.

Service: The configured listener was TCP/2222 on 0.0.0.0, and Cowrie reported that it was ready to accept SSH connections.

Evidence



```
2 pycparser-3.0 pyopenssl-26.4.0 service_identity-26.1.0 tonli-2.4.1 treq-26.7.0 twisted-26.4.0 typing-extensions-4.16.0 zope-interf
ace-8.6
(cowrie-env) cowrie@honeypot:~/network-honeypot$ cowrie init
Wrote etc/cowrie.cfg
Created var/log/cowrie, var/lib/cowrie, var/lib/cowrie/downloads, var/lib/cowrie/tty, var/run
Edit etc/cowrie.cfg to customize hostname, ports, etc., then run 'cowrie start'.
(cowrie-env) cowrie@honeypot:~/network-honeypot$ ls -lah
total 20K
drwxrwxr-x 5 cowrie cowrie 4.0K Sep 11 09:39 .
drwxr-x--- 4 cowrie cowrie 4.0K Sep 11 09:37 ..
drwxrwxr-x 5 cowrie cowrie 4.0K Sep 11 09:35 cowrie-env
drwxrwxr-x 2 cowrie cowrie 4.0K Sep 11 09:39 etc
drwxrwxr-x 5 cowrie cowrie 4.0K Sep 11 09:39 var
(cowrie-env) cowrie@honeypot:~/network-honeypot$ find var -maxdepth 2 -type d
var
var/lib
var/lib/cowrie
var/log
var/log/cowrie
var/run
(cowrie-env) cowrie@honeypot:~/network-honeypot$ ls -lah etc/
total 56K
drwxrwxr-x 2 cowrie cowrie 4.0K Sep 11 09:39 .
drwxrwxr-x 5 cowrie cowrie 4.0K Sep 11 09:39 ..
-rw-rw-r-- 1 cowrie cowrie 47K Sep 11 09:39 cowrie.cfg
(cowrie-env) cowrie@honeypot:~/network-honeypot$ grep -n "listen_endpoints" etc/cowrie.cfg
740:# listen_endpoints = tcp:6415:interface=127.0.0.1
744:# (default: listen_endpoints = tcp:2222:interface=0.0.0.0)
746:# listen_endpoints = systemd:domain=INET:index=0
747:# For both IPv4 and IPv6: listen_endpoints = tcp6:2223:interface=\:
749:# e.g listen_endpoints = "tcp:2222:interface=0.0.0.0 tcp:1022:interface=0.0.0.0" will result listening both on ports 2222 and 10
22
754:# the proxy's, e.g. listen_endpoints = haproxy:tcp:2222:interface=0.0.0.0
758:# listen_endpoints = tcp:2222:interface=0.0.0.0
831:# (default: listen_endpoints = tcp:2223:interface=0.0.0.0)
833:# listen_endpoints = systemd:domain=INET:index=0
834:# For IPv4 and IPv6: listen_endpoints = tcp6:2223:interface=\: tcp:2223:interface=0.0.0.0
836:# e.g "listen_endpoints = tcp:2223:interface=0.0.0.0 tcp:2323:interface=0.0.0.0" will result listening both on ports 2223 and 23
23
840:# listen_endpoints = haproxy:tcp:2223:interface=0.0.0.0
```

Figure 3.1 — Cowrie installation and configuration initialization

```
Activities Terminal Sep 11 09:45
cowrie@honeypot: ~/network-honeypot

834:# For IPv4 and IPv6: listen_endpoints = tcp6:2223:interface=:\: tcp:2223:interface=0.0.0.0
836:# e.g "listen_endpoints = tcp:2223:interface=0.0.0.0 tcp:2323:interface=0.0.0.0" will result listening both on ports 2223 and 23
23
840:# listen_endpoints = haproxy:tcp:2223:interface=0.0.0.0
842:listen_endpoints = tcp:2223:interface=0.0.0.0
(cowrie-env) cowrie@honeypot:~/network-honeypot$ ~/network-honeypot
-bash: /home/cowrie/network-honeypot: Is a directory
(cowrie-env) cowrie@honeypot:~/network-honeypot$ cowrie start

Join the Cowrie community at: https://www.cowrie.org/slack/

Starting cowrie: [twistd --umask=0022 --pidfile var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie]...
(cowrie-env) cowrie@honeypot:~/network-honeypot$ cowrie status
cowrie is running (PID: 7112).
(cowrie-env) cowrie@honeypot:~/network-honeypot$ ss -tulnp | grep 2222
tcp LISTEN 0 50 0.0.0.0:2222 0.0.0.0:* users:((("twistd",pid=7112,fd=11))
(cowrie-env) cowrie@honeypot:~/network-honeypot$ ss -tln | grep 2222
LISTEN 0 50 0.0.0.0:2222 0.0.0.0:*
(cowrie-env) cowrie@honeypot:~/network-honeypot$ ls -lah var/log/cowrie/
total 12K
drwxrwxr-x 2 cowrie cowrie 4.0K Sep 11 09:42 .
drwxrwxr-x 3 cowrie cowrie 4.0K Sep 11 09:39 ..
-rw-rw-r-- 1 cowrie cowrie 0 Sep 11 09:42 cowrie.json
-rw-rw-r-- 1 cowrie cowrie 1.2K Sep 11 09:42 cowrie.log
(cowrie-env) cowrie@honeypot:~/network-honeypot$ tail -n 20 var/log/cowrie/cowrie.log
2026-09-11T09:42:36.990887Z [cowrie.core.config#info] Reading configuration from ['etc/cowrie.cfg']
2026-09-11T09:42:37.207368Z [cowrie.plugin#info] Python Version 3.10.12 (main, Aug 31 2026, 10:18:17) [GCC 11.4.0]
2026-09-11T09:42:37.207389Z [cowrie.plugin#info] Twisted Version 26.4.0
2026-09-11T09:42:37.207396Z [cowrie.plugin#info] Cowrie Version 3.0.13
2026-09-11T09:42:37.207402Z [cowrie.plugin#info] Sensor UUID: 05a98f7a-ad97-11f1-b672-313b5079f9e5
2026-09-11T09:42:37.207819Z [cowrie.core.output#info] Loaded output engine: jsonlog
2026-09-11T09:42:37.208686Z [twisted.scripts._twistd_unix.UnixAppLogger#info] twistd 26.4.0 (/home/cowrie/network-honeypot/cowrie-en
v/bin/python 3.10.12) starting up.
2026-09-11T09:42:37.208763Z [twisted.scripts._twistd_unix.UnixAppLogger#info] reactor class: twisted.internet.epollreactor.EPollReac
tor.
2026-09-11T09:42:37.214200Z [-] CowrieSSHFactory starting on 2222
2026-09-11T09:42:37.214832Z [cowrie.ssh.factory.CowrieSSHFactory#info] Starting factory <cowrie.ssh.factory.CowrieSSHFactory object
at 0x7446749fe560>
2026-09-11T09:42:37.303537Z [cowrie.ssh.factory.CowrieSSHFactory#info] Ready to accept SSH connections
(cowrie-env) cowrie@honeypot:~/network-honeypot$
```

Figure 3.2 — Cowrie service startup and readiness for SSH connections

Module result

Cowrie was installed and configured successfully and exposed an SSH honeypot listener on TCP/2222.

PRACTICAL MODULE 4 — CONTROLLED SSH INTERACTION & LOGGING

Objective: Connect from Kali to Cowrie using a laboratory-only test account, execute safe shell commands and verify that Cowrie records authentication, commands, sessions and termination.

Connection: Kali connected to the Cowrie service at 10.208.147.139:2222 using the laboratory test account testuser.

Interaction: Safe commands such as whoami, pwd, ls, uname -a and id were executed inside the simulated shell.

Logging: Cowrie recorded the source IP, authentication activity, commands, session state and termination.

Evidence

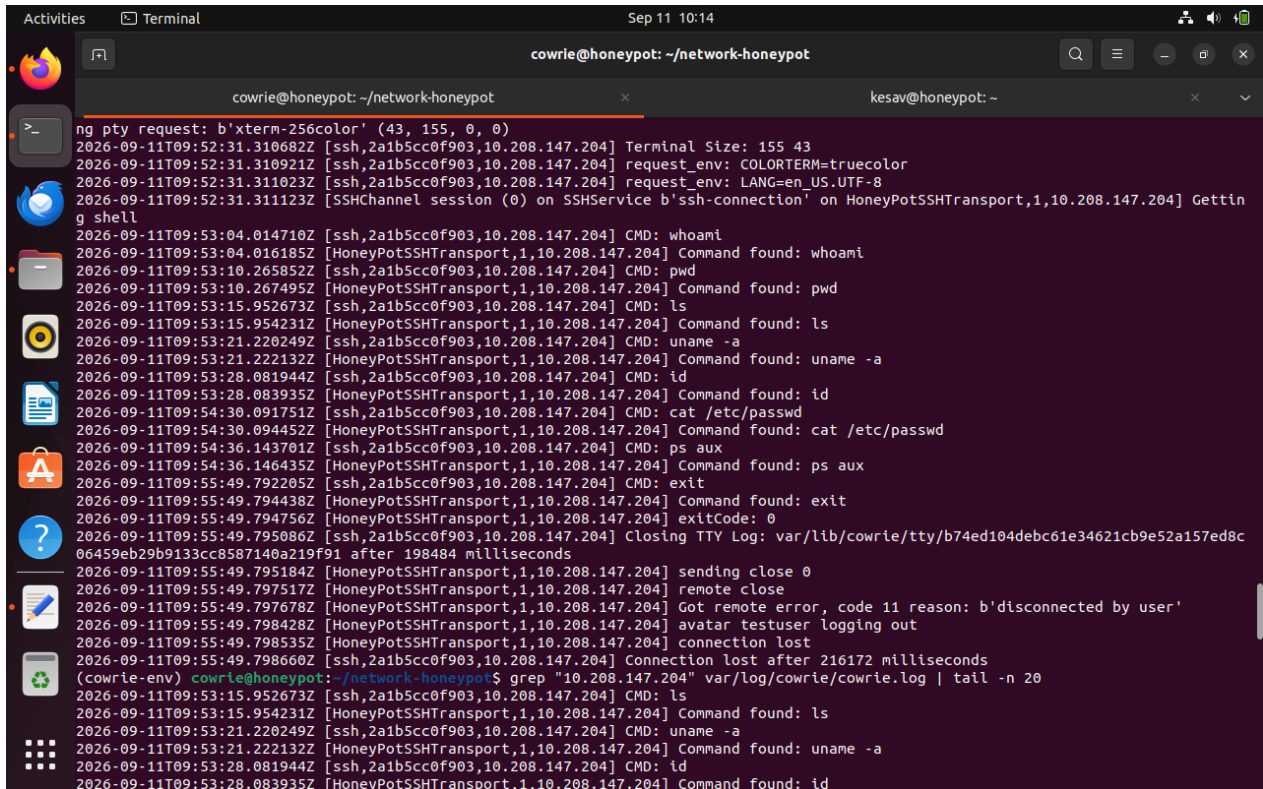


```
(kali@kali)~$ ssh -p 2222 testuser@10.208.147.139
The authenticity of host '[10.208.147.139]:2222 ([10.208.147.139]:2222)' can't be established.
ED25519 key fingerprint is: SHA256:YfHUXemHoim6EPifcJTj/wm1SpFXNuu4w9+cK+FZWHI
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[10.208.147.139]:2222' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
testuser@10.208.147.139's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
testuser@svr04:~$ whoami
testuser
testuser@svr04:~$ pwd
/home/testuser
testuser@svr04:~$ ls
testuser@svr04:~$ uname -a
Linux svr04 6.1.0-21-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.90-1 (2024-05-03) x86_64 GNU/Linux
testuser@svr04:~$ id
uid=8597(testuser) gid=8597(testuser) groups=8597(testuser)
testuser@svr04:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
```

Figure 4.1 — Controlled SSH session through Cowrie



```
ng pty request: b'xterm-256color' (43, 155, 0, 0)
2026-09-11T09:52:31.310682Z [ssh,2a1b5cc0f903,10.208.147.204] Terminal Size: 155 43
2026-09-11T09:52:31.310921Z [ssh,2a1b5cc0f903,10.208.147.204] request_env: COLORTERM=truecolor
2026-09-11T09:52:31.311023Z [ssh,2a1b5cc0f903,10.208.147.204] request_env: LANG=en_US.UTF-8
2026-09-11T09:52:31.311123Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransport,1,10.208.147.204] Gettin
g shell
2026-09-11T09:53:04.014710Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: whoami
2026-09-11T09:53:04.016185Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: whoami
2026-09-11T09:53:10.265852Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: pwd
2026-09-11T09:53:10.267495Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: pwd
2026-09-11T09:53:15.952673Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: ls
2026-09-11T09:53:15.954231Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: ls
2026-09-11T09:53:21.220249Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: uname -a
2026-09-11T09:53:21.222132Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: uname -a
2026-09-11T09:53:28.081944Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: id
2026-09-11T09:53:28.083935Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: id
2026-09-11T09:54:30.091751Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: cat /etc/passwd
2026-09-11T09:54:30.094452Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: cat /etc/passwd
2026-09-11T09:54:36.143701Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: ps aux
2026-09-11T09:54:36.146435Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: ps aux
2026-09-11T09:55:49.792205Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: exit
2026-09-11T09:55:49.794438Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: exit
2026-09-11T09:55:49.794756Z [HoneyPotSSHTransport,1,10.208.147.204] exitCode: 0
2026-09-11T09:55:49.795086Z [ssh,2a1b5cc0f903,10.208.147.204] Closing TTY Log: var/lib/cowrie/tty/b74ed104debc61e34621cb9e52a157ed8c
06459eb29b9133cc8587140a219f91 after 198484 milliseconds
2026-09-11T09:55:49.795184Z [HoneyPotSSHTransport,1,10.208.147.204] sending close 0
2026-09-11T09:55:49.797517Z [HoneyPotSSHTransport,1,10.208.147.204] remote close
2026-09-11T09:55:49.797678Z [HoneyPotSSHTransport,1,10.208.147.204] Got remote error, code 11 reason: b'disconnected by user'
2026-09-11T09:55:49.798428Z [HoneyPotSSHTransport,1,10.208.147.204] avatar testuser logging out
2026-09-11T09:55:49.798535Z [HoneyPotSSHTransport,1,10.208.147.204] connection lost
2026-09-11T09:55:49.798660Z [ssh,2a1b5cc0f903,10.208.147.204] Connection lost after 216172 milliseconds
(cowrie-env) cowrie@honeypot:~/network-honeypot$ grep "10.208.147.204" var/log/cowrie/cowrie.log | tail -n 20
2026-09-11T09:53:15.952673Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: ls
2026-09-11T09:53:15.954231Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: ls
2026-09-11T09:53:21.220249Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: uname -a
2026-09-11T09:53:21.222132Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: uname -a
2026-09-11T09:53:28.081944Z [ssh,2a1b5cc0f903,10.208.147.204] CMD: id
2026-09-11T09:53:28.083935Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: id
```

Figure 4.2 — Cowrie records commands and session activity

Module result

Controlled SSH interaction was captured by Cowrie, including authentication, commands and session lifecycle information.

PRACTICAL MODULE 5 — CONTROLLED NMAP RECONNAISSANCE

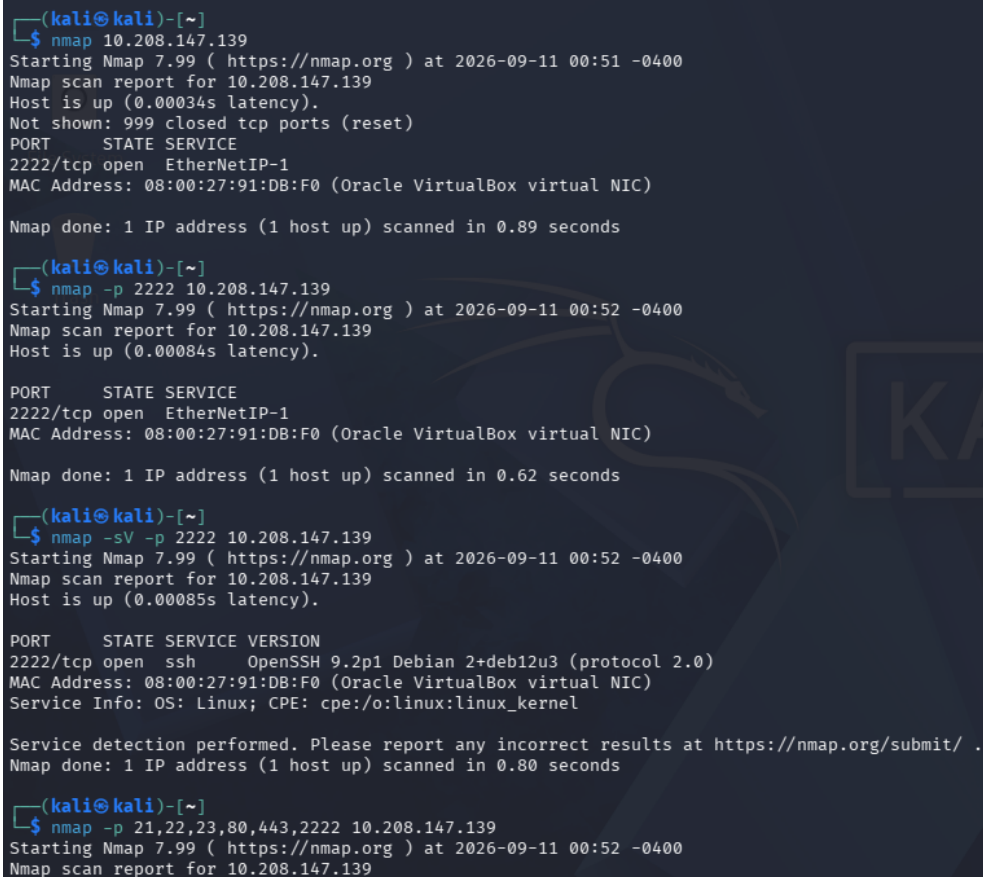
Objective: Perform reconnaissance only against the laboratory honeypot to identify reachable ports and service information, and preserve the Nmap output as evidence.

Host discovery: Nmap confirmed that the honeypot host was reachable.

Port reconnaissance: TCP/2222 was identified as open while selected common ports were observed as closed.

Service detection: A version-detection scan identified an SSH service on TCP/2222.

Evidence



```
(kali㉿kali)-[~]
└─$ nmap 10.208.147.139
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 00:51 -0400
Nmap scan report for 10.208.147.139
Host is up (0.00034s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
2222/tcp  open  EtherNetIP-1
MAC Address: 08:00:27:91:DB:F0 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.89 seconds

(kali㉿kali)-[~]
└─$ nmap -p 2222 10.208.147.139
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 00:52 -0400
Nmap scan report for 10.208.147.139
Host is up (0.00084s latency).

PORT      STATE SERVICE
2222/tcp  open  EtherNetIP-1
MAC Address: 08:00:27:91:DB:F0 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.62 seconds

(kali㉿kali)-[~]
└─$ nmap -sV -p 2222 10.208.147.139
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 00:52 -0400
Nmap scan report for 10.208.147.139
Host is up (0.00085s latency).

PORT      STATE SERVICE VERSION
2222/tcp  open  ssh      OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
MAC Address: 08:00:27:91:DB:F0 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.80 seconds

(kali㉿kali)-[~]
└─$ nmap -p 21,22,23,80,443,2222 10.208.147.139
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 00:52 -0400
Nmap scan report for 10.208.147.139
```

Figure 5.1 — Nmap discovery and service/version reconnaissance

```

(kali@kali) [~]
$ nmap -p 21,22,23,80,443,2222 10.208.147.139
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 00:52 -0400
Nmap scan report for 10.208.147.139
Host is up (0.0011s latency).

PORT      STATE SERVICE
21/tcp    closed ftp
22/tcp    closed ssh
23/tcp    closed telnet
80/tcp    closed http
443/tcp   closed https
2222/tcp  open  EtherNetIP-1
MAC Address: 08:00:27:91:DB:F0 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.61 seconds

(kali@kali) [~]
$ nmap -sT -p 2222 10.208.147.139
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 00:52 -0400
Nmap scan report for 10.208.147.139
Host is up (0.00082s latency).

PORT      STATE SERVICE
2222/tcp  open  EtherNetIP-1
MAC Address: 08:00:27:91:DB:F0 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.57 seconds

(kali@kali) [~]
$ nmap -p 2222 10.208.147.139
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 00:54 -0400
Nmap scan report for 10.208.147.139
Host is up (0.00079s latency).

PORT      STATE SERVICE
2222/tcp  open  EtherNetIP-1
MAC Address: 08:00:27:91:DB:F0 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds

```

Figure 5.2 — Controlled port-state scan against the honeypot

Module result

Nmap successfully demonstrated controlled reconnaissance and identified TCP/2222 as an open SSH service.

PRACTICAL MODULE 6 — WIRESHARK NETWORK TRAFFIC ANALYSIS

Objective: Capture and inspect packet-level traffic for the Cowrie service, including the TCP three-way handshake, SSHv2 negotiation, encrypted traffic and source/destination information.

Capture: Wireshark captured traffic on Kali's eth0 interface.

Handshake: The TCP three-way handshake to port 2222 was observed as SYN, SYN/ACK and ACK.

SSH analysis: SSHv2 protocol negotiation, key exchange and encrypted packets were visible.

Correlation: The source and destination addresses matched the laboratory IPs used in the Cowrie logs.

Evidence

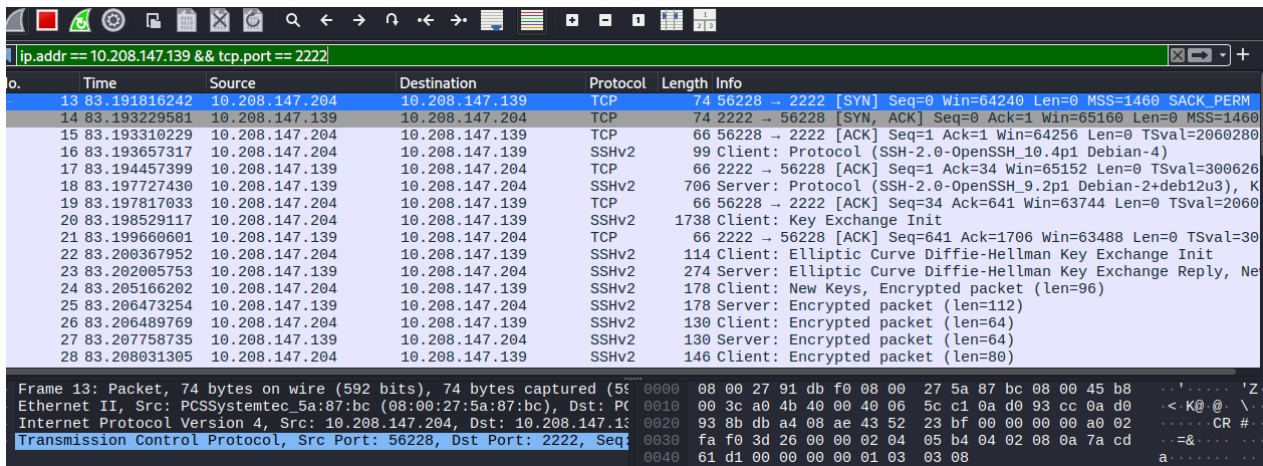


Figure 6.1 — TCP three-way handshake to Cowrie port 2222

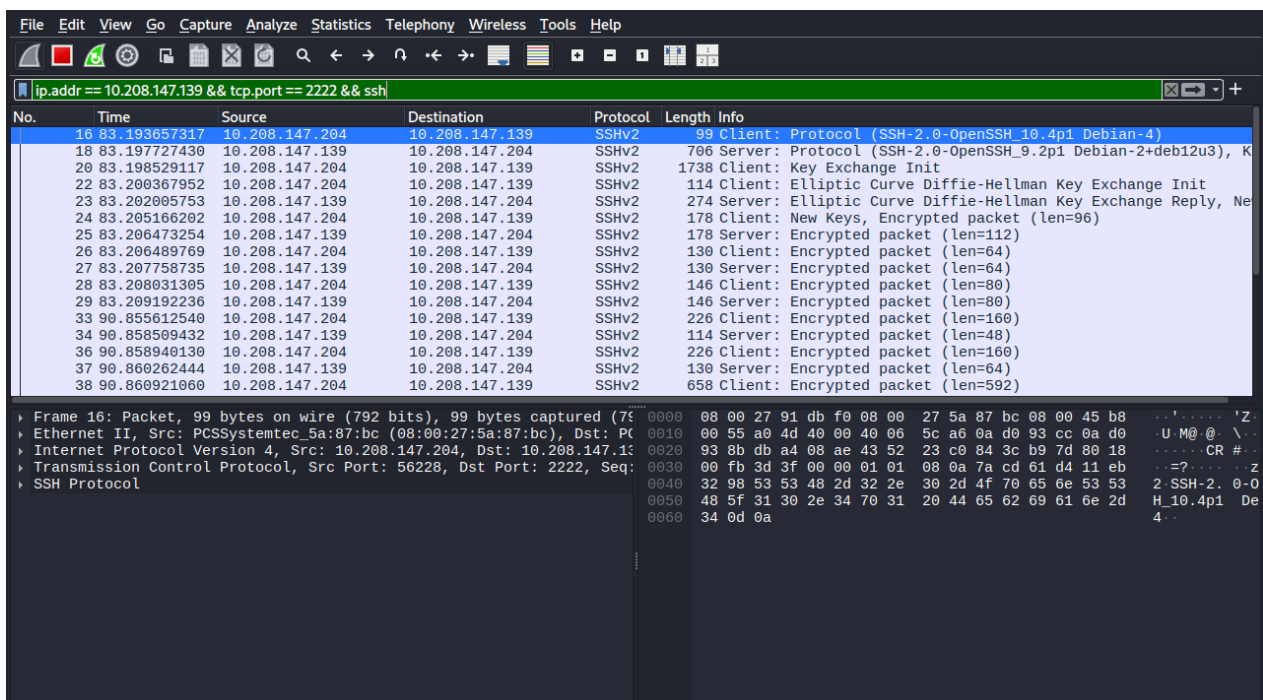


Figure 6.2 — SSHv2 negotiation and encrypted traffic

```

▶ Frame 13: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on inte
▶ Ethernet II, Src: PCSSystemtec_5a:87:bc (08:00:27:5a:87:bc), Dst: PCSSystemtec_91:d
▼ Internet Protocol Version 4, Src: 10.208.147.204, Dst: 10.208.147.139
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0xb8 (DSCP: EF, ECN: Not-ECT)
    Total Length: 60
    Identification: 0xa04b (41035)
    ▶ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0x5cc1 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.208.147.204
    Destination Address: 10.208.147.139
    [Stream index: 1]
    Transmission Control Protocol, Src Port: 56220, Dest Port: 2222, Seq: 0, Len: 0

```

Figure 6.3 — IPv4 and TCP source/destination details

Module result

Wireshark provided packet-level evidence of the TCP handshake, SSHv2 negotiation, encrypted traffic and endpoint/port details.

PRACTICAL MODULE 7 — LOG ANALYSIS & EVENT CORRELATION

Objective: Analyze Cowrie text and JSON logs, identify source IP, authentication activity, commands and session events, then correlate application-level evidence with Wireshark packet evidence.

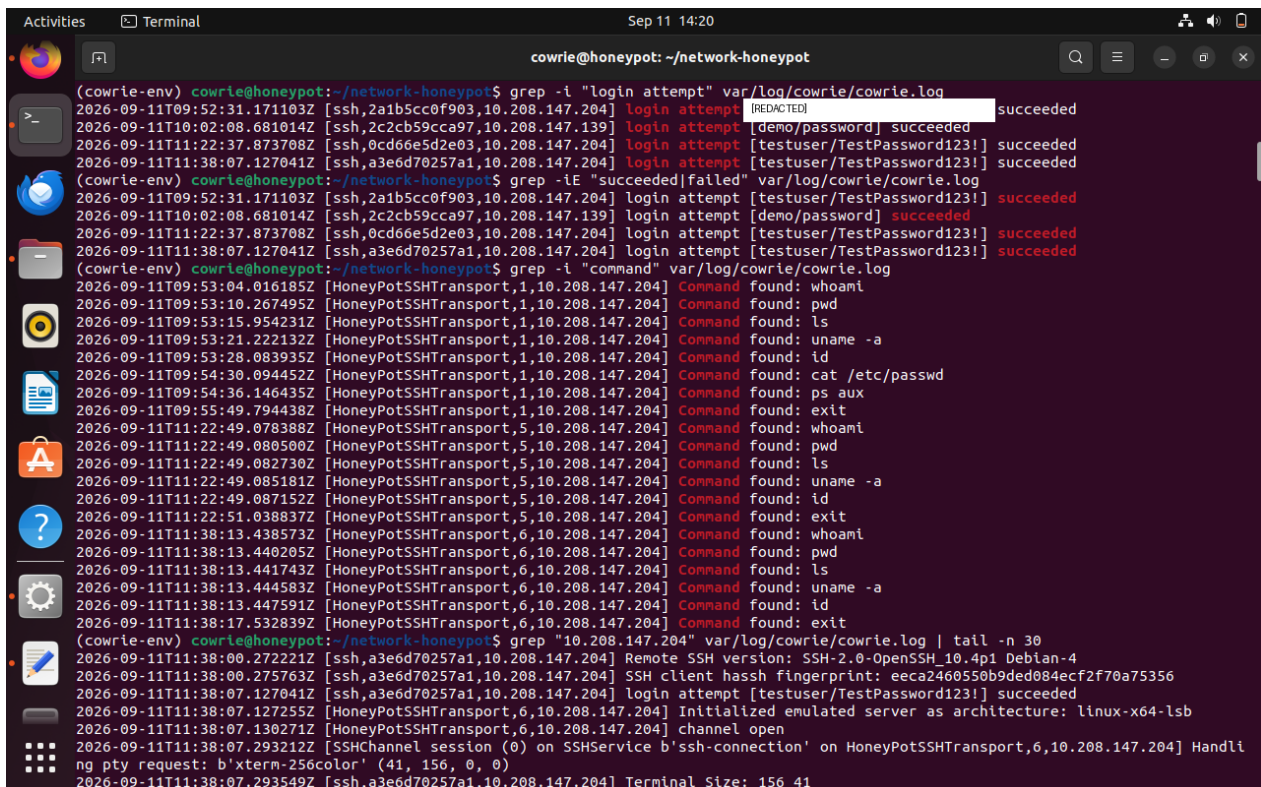
Text logs: Cowrie activity logs were reviewed for connections, login attempts, commands and session termination.

JSON logs: cowrie.json was reviewed as a structured event source.

Source identification: The Kali source IP 10.208.147.204 was correlated with the honeypot destination 10.208.147.139.

Timeline: The evidence supports a sequence from connection and SSH negotiation through authentication, command execution and session termination.

Evidence



```
Activities Terminal Sep 11 14:20
cowrie@honeypot: ~/network-honeypot

(cowrie-env) cowrie@honeypot:~/network-honeypot$ grep -i "login attempt" var/log/cowrie/cowrie.log
2026-09-11T09:52:31.171103Z [ssh,2a1b5cc0f903,10.208.147.204] login attempt [REDACTED] succeeded
2026-09-11T10:02:08.681014Z [ssh,2c2cb59cca97,10.208.147.139] login attempt [demo/password] succeeded
2026-09-11T11:22:37.873708Z [ssh,0cd66e5d2e03,10.208.147.204] login attempt [testuser/TestPassword123!] succeeded
2026-09-11T11:38:07.127041Z [ssh,a3e6d70257a1,10.208.147.204] login attempt [testuser/TestPassword123!] succeeded

(cowrie-env) cowrie@honeypot:~/network-honeypot$ grep -iE "succeeded|failed" var/log/cowrie/cowrie.log
2026-09-11T09:52:31.171103Z [ssh,2a1b5cc0f903,10.208.147.204] login attempt [testuser/TestPassword123!] succeeded
2026-09-11T10:02:08.681014Z [ssh,2c2cb59cca97,10.208.147.139] login attempt [demo/password] succeeded
2026-09-11T11:22:37.873708Z [ssh,0cd66e5d2e03,10.208.147.204] login attempt [testuser/TestPassword123!] succeeded
2026-09-11T11:38:07.127041Z [ssh,a3e6d70257a1,10.208.147.204] login attempt [testuser/TestPassword123!] succeeded

(cowrie-env) cowrie@honeypot:~/network-honeypot$ grep -i "command" var/log/cowrie/cowrie.log
2026-09-11T09:53:04.016185Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: whoami
2026-09-11T09:53:10.267495Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: pwd
2026-09-11T09:53:15.954231Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: ls
2026-09-11T09:53:21.222132Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: uname -a
2026-09-11T09:53:28.083935Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: id
2026-09-11T09:54:30.094452Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: cat /etc/passwd
2026-09-11T09:54:36.146435Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: ps aux
2026-09-11T09:55:49.794438Z [HoneyPotSSHTransport,1,10.208.147.204] Command found: exit
2026-09-11T11:22:49.078388Z [HoneyPotSSHTransport,5,10.208.147.204] Command found: whoami
2026-09-11T11:22:49.080500Z [HoneyPotSSHTransport,5,10.208.147.204] Command found: pwd
2026-09-11T11:22:49.082730Z [HoneyPotSSHTransport,5,10.208.147.204] Command found: ls
2026-09-11T11:22:49.085181Z [HoneyPotSSHTransport,5,10.208.147.204] Command found: uname -a
2026-09-11T11:22:49.087152Z [HoneyPotSSHTransport,5,10.208.147.204] Command found: id
2026-09-11T11:22:51.038837Z [HoneyPotSSHTransport,5,10.208.147.204] Command found: exit
2026-09-11T11:38:13.438573Z [HoneyPotSSHTransport,6,10.208.147.204] Command found: whoami
2026-09-11T11:38:13.440205Z [HoneyPotSSHTransport,6,10.208.147.204] Command found: pwd
2026-09-11T11:38:13.441743Z [HoneyPotSSHTransport,6,10.208.147.204] Command found: ls
2026-09-11T11:38:13.444583Z [HoneyPotSSHTransport,6,10.208.147.204] Command found: uname -a
2026-09-11T11:38:13.447591Z [HoneyPotSSHTransport,6,10.208.147.204] Command found: id
2026-09-11T11:38:17.532839Z [HoneyPotSSHTransport,6,10.208.147.204] Command found: exit

(cowrie-env) cowrie@honeypot:~/network-honeypot$ grep "10.208.147.204" var/log/cowrie/cowrie.log | tail -n 30
2026-09-11T11:38:00.272221Z [ssh,a3e6d70257a1,10.208.147.204] Remote SSH version: SSH-2.0-OpenSSH_10.4p1 Debian-4
2026-09-11T11:38:00.275763Z [ssh,a3e6d70257a1,10.208.147.204] SSH client hassh fingerprint: eeca2460550b9ded084ecf2f70a75356
2026-09-11T11:38:07.127041Z [ssh,a3e6d70257a1,10.208.147.204] login attempt [testuser/TestPassword123!] succeeded
2026-09-11T11:38:07.127255Z [HoneyPotSSHTransport,6,10.208.147.204] Initialized emulated server as architecture: linux-x64-lsb
2026-09-11T11:38:07.130271Z [HoneyPotSSHTransport,6,10.208.147.204] channel open
2026-09-11T11:38:07.293212Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransport,6,10.208.147.204] Handling
pty request: b'xterm-256color' (41, 156, 0, 0)
2026-09-11T11:38:07.293549Z [ssh,a3e6d70257a1,10.208.147.204] Terminal Size: 156 41
```

Figure 7.1 — Cowrie authentication and command events

```
Activities Terminal Sep 11 14:20
cowrie@honeypot: ~/network-honeypot

6-09-11T11:38:17.537376Z", "message": "Connection lost after 17265 milliseconds"}
(cowrie-env) cowrie@honeypot:~/network-honeypot$ exit
logout
kesav@kesav-VirtualBox:~$ sudo apt install -y jq
[sudo] password for kesav:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.6-2.1ubuntu3.2).
jq set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
kesav@kesav-VirtualBox:~$ sudo su - cowrie
cowrie@honeypot:~$ cd ~/network-honeypot
cowrie@honeypot:~/network-honeypot$ jq . var/log/cowrie/cowrie.json | tail -n 100
{
  "protocol": "ssh",
  "src_ip": "10.208.147.204",
  "src_port": 47588,
  "dst_ip": "10.208.147.139",
  "dst_port": 2222,
  "input": "pwd",
  "eventid": "cowrie.command.input",
  "sensor": "honeypot",
  "uuid": "05a98f7a-ad97-11f1-b672-313b5079f9e5",
  "timestamp": "2026-09-11T11:38:13.439054Z",
  "message": "CMD: pwd"
}
{
  "session": "a3e6d70257a1",
  "protocol": "ssh",
  "src_ip": "10.208.147.204",
  "src_port": 47588,
  "dst_ip": "10.208.147.139",
  "dst_port": 2222,
  "input": "ls",
  "eventid": "cowrie.command.input",
  "sensor": "honeypot",
  "uuid": "05a98f7a-ad97-11f1-b672-313b5079f9e5",
  "timestamp": "2026-09-11T11:38:13.440518Z",
  "message": "CMD: ls"
}
```

Figure 7.2 — Structured Cowrie JSON events

```
Activities Terminal Sep 11 14:21
cowrie@honeypot: ~/network-honeypot

SSHv2 negotiation observed
Encrypted SSH traffic observed

Analysis:
Cowrie provides application/session-level evidence.
Wireshark provides packet-level network evidence.
The two sources can be correlated using source IP,
destination IP, port, timestamps and session activity.
EOF
cowrie@honeypot:~/network-honeypot$ cat ~/network-honeypot/module8_summary.txt
MODULE 8 - LOG ANALYSIS SUMMARY

Honeypot IP: 10.208.147.139
Attacker/Test IP: 10.208.147.204
Honeypot Service: Cowrie SSH
SSH Port: 2222

Authentication:
Username tested: testuser
Authentication result: Successful

Commands observed:
whoami
pwd
ls
uname -a
id
exit

Network Evidence:
TCP connection to port 2222
SSHv2 negotiation observed
Encrypted SSH traffic observed

Analysis:
Cowrie provides application/session-level evidence.
Wireshark provides packet-level network evidence.
The two sources can be correlated using source IP,
destination IP, port, timestamps and session activity.
cowrie@honeypot:~/network-honeypot$
```

Figure 7.3 — Cowrie and network-evidence correlation summary

Module result

Cowrie text and JSON logs were analyzed and correlated with network evidence to construct a coherent event timeline.

PRACTICAL MODULE 8 — RESULTS, ATTACK ANALYSIS & FINDINGS

Objective: Consolidate the experimental observations into security findings, attack sequence, detection observations, limitations and the overall outcome of the honeypot experiment.

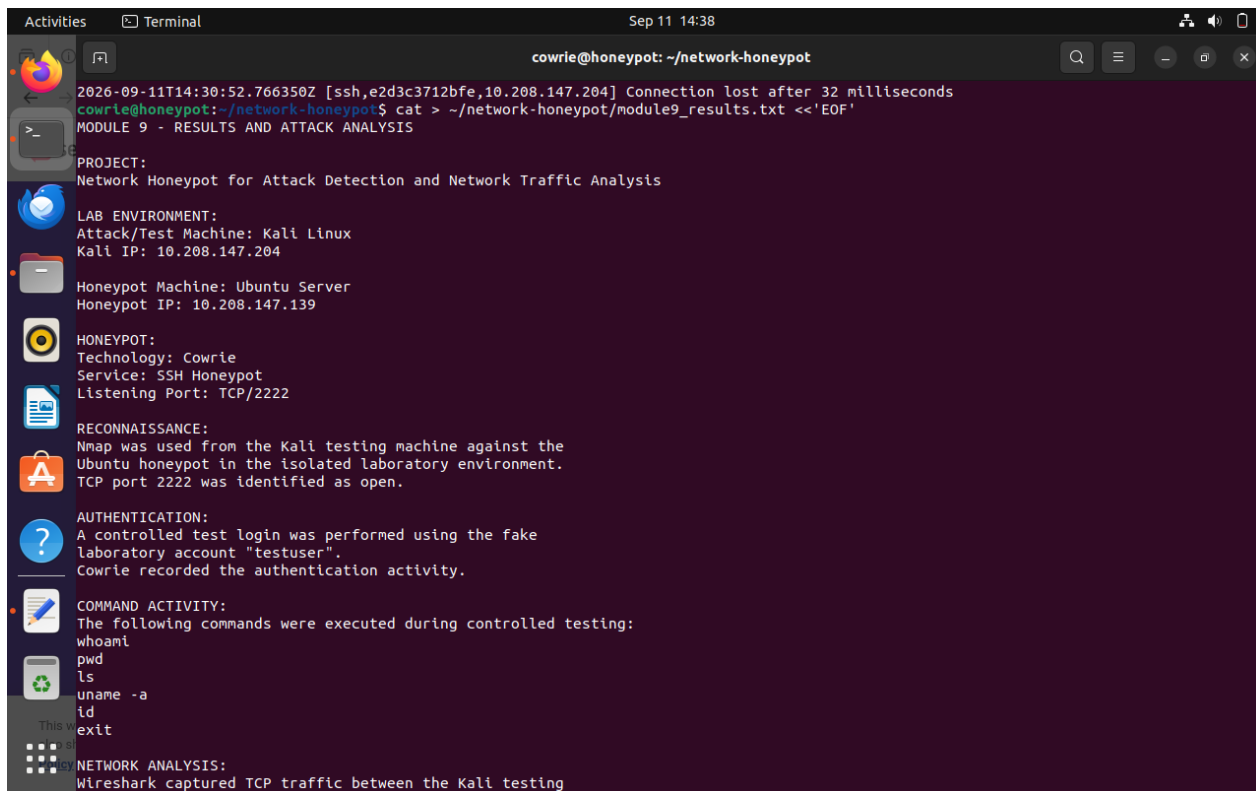
Reconnaissance result: Controlled Nmap testing identified TCP/2222 as the reachable honeypot service.

SSH result: Cowrie successfully recorded controlled authentication and interactive shell activity.

Packet result: Wireshark captured the TCP handshake and SSHv2/encrypted traffic.

Overall finding: Combining Cowrie and Wireshark provides complementary application-level and packet-level visibility into SSH activity.

Evidence



```
2026-09-11T14:30:52.766350Z [ssh,e2d3c3712bfe,10.208.147.204] Connection lost after 32 milliseconds
cowrie@honeypot:~/network-honeypot$ cat > ~/network-honeypot/module9_results.txt <<'EOF'
MODULE 9 - RESULTS AND ATTACK ANALYSIS

PROJECT:
Network Honeypot for Attack Detection and Network Traffic Analysis

LAB ENVIRONMENT:
Attack/Test Machine: Kali Linux
Kali IP: 10.208.147.204

Honeypot Machine: Ubuntu Server
Honeypot IP: 10.208.147.139

HONEYPOT:
Technology: Cowrie
Service: SSH Honeypot
Listening Port: TCP/2222

RECONNAISSANCE:
Nmap was used from the Kali testing machine against the
Ubuntu honeypot in the isolated laboratory environment.
TCP port 2222 was identified as open.

AUTHENTICATION:
A controlled test login was performed using the fake
laboratory account "testuser".
Cowrie recorded the authentication activity.

COMMAND ACTIVITY:
The following commands were executed during controlled testing:
whoami
pwd
ls
uname -a
id
exit

NETWORK ANALYSIS:
Wireshark captured TCP traffic between the Kali testing
```

Figure 8.1 — Consolidated project results

```
Activities Terminal Sep 11 14:38
cowrie@honeypot: ~/network-honeypot

interaction through Cowrie. Additional protocols were
not deployed because the OpenCanary module was skipped.
Testing was performed only within the isolated laboratory.
cowrie@honeypot:~/network-honeypot$ cat > ~/network-honeypot/findings.txt <<'EOF'
=====
NETWORK HONEYPOT - SECURITY FINDINGS
=====
Finding 1: Open SSH Honeypot Port
Evidence: Nmap + Wireshark
Port: TCP/2222
Result: Accessible from Kali testing machine

Finding 2: SSH Connection Detection
Evidence: Cowrie + Wireshark
Source: 10.208.147.204
Destination: 10.208.147.139:2222
Result: Connection recorded

Finding 3: Authentication Activity
Evidence: Cowrie logs
Username: testuser
Result: Controlled authentication recorded

Finding 4: Interactive Command Activity
Evidence: Cowrie logs
Commands:
whoami
pwd
ls
uname -a
id
exit

Finding 5: Network-Level Evidence
Evidence: Wireshark
Result:
TCP three-way handshake observed
SSHv2 negotiation observed
Encrypted SSH traffic observed
```

Figure 8.2 — Security findings and overall result

```
Activities Terminal Sep 11 14:38
cowrie@honeypot: ~/network-honeypot

controlled SSH-based attack/test activity and provided
both application-level and packet-level evidence.
=====
cowrie@honeypot:~/network-honeypot$ echo "===== PROJECT EVIDENCE CHECK ====="
echo "[1] Cowrie logs:"
ls -lh ~/network-honeypot/var/log/cowrie/

echo
echo "[2] Cowrie status:"
cowrie status

echo
echo "[3] Cowrie port:"
ss -tln | grep 2222

echo
echo "[4] Results file:"
ls -lh ~/network-honeypot/module9_results.txt

echo
echo "[5] Findings file:"
ls -lh ~/network-honeypot/findings.txt
===== PROJECT EVIDENCE CHECK =====
[1] Cowrie logs:
total 64K
-rw-rw-r-- 1 cowrie cowrie 33K Sep 11 14:30 cowrie.json
-rw-rw-r-- 1 cowrie cowrie 17K Sep 11 14:30 cowrie.log

[2] Cowrie status:
cowrie: command not found

[3] Cowrie port:
LISTEN 0      50          0.0.0.0:2222  0.0.0.0:*

[4] Results file:
-rw-rw-r-- 1 cowrie cowrie 1.6K Sep 11 14:36 /home/cowrie/network-honeypot/module9_results.txt

[5] Findings file:
-rw-rw-r-- 1 cowrie cowrie 1.3K Sep 11 14:37 /home/cowrie/network-honeypot/findings.txt
cowrie@honeypot:~/network-honeypot$
```

Figure 8.3 — Final project evidence verification

Module result

The consolidated results demonstrate that the honeypot provides useful defensive visibility into controlled SSH reconnaissance and interaction.

17. OVERALL RESULTS

The completed experiment demonstrates a functioning SSH honeypot and an evidence-driven analysis workflow. Kali at 10.208.147.204 reached Ubuntu at 10.208.147.139; Cowrie listened on TCP/2222 and recorded controlled SSH activity; Wireshark captured the corresponding network communication.

Observation	Evidence	Result
Honeypot reachability	Nmap / nc / ping	Verified
SSH service exposure	Nmap / ss	TCP/2222 open and listening
Authentication activity	Cowrie logs	Recorded
Interactive commands	Cowrie logs / JSON	Recorded
TCP handshake	Wireshark	Observed
SSHv2 negotiation	Wireshark	Observed
Encrypted SSH traffic	Wireshark	Observed
Source IP correlation	Cowrie + Wireshark	10.208.147.204 correlated to 10.208.147.139

18. SECURITY ANALYSIS AND DISCUSSION

Cowrie provides application/session-level context such as authentication attempts and shell commands, while Wireshark provides packet-level evidence of the network path, TCP behavior and SSHv2 exchange. Correlating the same source/destination IPs, TCP/2222 and timestamps produces a more defensible event narrative.

Attack sequence

- Reconnaissance
- TCP connection to 2222
- TCP handshake
- SSHv2 negotiation and key exchange
- Authentication
- Interactive commands
- Session termination

19. LIMITATIONS

- Academic isolated laboratory only
- SSH-focused implementation
- OpenCanary services not included in the eight practical modules
- Small, safe command set
- Private IP addressing and laboratory credentials
- Encrypted SSH contents are not directly visible in Wireshark

20. FUTURE SCOPE

- Add OpenCanary and additional protocol honeypots
- Forward Cowrie events to Splunk/ELK/SIEM
- Automate alerts for repeated authentication failures and reconnaissance
- Add dashboards and long-term event storage
- Expand to multiple honeypot nodes

- Add controlled file/malware research in a properly isolated environment

21. CONCLUSION

The project successfully demonstrates a network honeypot for attack detection and network traffic analysis. Cowrie captured controlled SSH authentication and shell activity, while Wireshark provided packet-level evidence of TCP and SSH behavior. The correlation of these sources produced a clear attack timeline and demonstrated the defensive value of a monitored decoy service.

22. REFERENCES

- Cowrie documentation and project repository — official open-source documentation.
- Nmap documentation — official Nmap reference and usage documentation.
- Wireshark documentation — official Wireshark User's Guide and display-filter documentation.
- Ubuntu Server documentation — official Ubuntu documentation.
- VirtualBox documentation — official Oracle VirtualBox documentation.

APPENDIX A — EVIDENCE MAPPING

Report Module	Evidence IDs	Description
M1	M1-01 to M1-06	IP configuration, ping and routing
M2	M2-01 to M2-06	Ubuntu preparation, packages, baseline and host state
M3	M3-01 to M3-13	Cowrie installation, configuration, startup and readiness
M4	M4-01 to M4-10	SSH connection, shell interaction and Cowrie logs
M5	M5-01 to M5-12	Nmap reconnaissance and packet evidence
M6	M7-01 to M7-10	Wireshark interface, handshake, SSH and traffic analysis
M7	M8-01 to M8-11	Cowrie text/JSON logs and event correlation
M8	M9-01 to M9-11	Results, findings and final evidence check

APPENDIX B — IMPORTANT LAB VALUES

- Kali Linux: 10.208.147.204
- Ubuntu honeypot: 10.208.147.139
- Private network: 10.208.147.0/24
- Cowrie SSH listener: TCP/2222
- Wireshark interface: eth0
- Laboratory account: testuser
- Passwords shown in screenshots should be redacted in any public submission.